Concurrency and Seat-Allocation Report

Deliverable 3 — Reliability and Concurrency

CS455 Campus Ride-Pooling System | PostgreSQL integration experiment passed

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

1. Objective and scope

This report documents the concurrency strategy used for ride seat allocation and the experiment implemented for the project's mandatory concurrency check (M17). The principal requirement is NFR-RD-05 in the Rider non-functional requirements:

> With at least 50 concurrent acceptance and leave operations on a ride with one available seat, repeated 100 times: zero over-allocations, and the seat count always equals capacity minus occupants.

The corresponding Ride Owner requirements are NFR-RO-DEP-01 (strong consistency between ride lifecycle and request decisions), NFR-RO-DEP-02 (concurrent joins and acceptances must not exceed fixed capacity), NFR-RO-DEP-03 (atomic persistence of request decisions and ride state), and NFR-RO-DEP-04 (locked rides reject normal membership changes).

The scope here is the implemented ride/request workflow in the Next.js application and PostgreSQL database. It does not claim to test a deployed multi-instance service, a 50-connection database pool, payment settlement, or physical vehicle allocation.

2. Shared state and correctness invariant

For a ride r, define:

$$O(r) = |\{s \in \texttt{riders} : s.\texttt{ride_id} = r.\texttt{id} \wedge s.\texttt{left_at IS NULL}\}|$$

and let $C(r)$ be `rides.capacity_snapshot`. The seat-allocation invariant is

$$1 \le O(r) \le C(r), \qquad \text{available}(r) = C(r) - O(r).$$

The owner is included among active occupants. Every other active occupant must have a corresponding join request in the `accepted` state. A rider who leaves remains in the `riders` history, but their non-null `left_at` means that they no longer consume a seat.

The contended resource is the ride row and its active occupant set. Two acceptance transactions can both read the same apparent free seat if the read/check/write sequence is not serialized. A second hazard is an acceptance racing with a leave: both operations change the same occupancy count, and their ordering determines how many seats are free at each point.

3. Implemented concurrency control

3.1 Transaction and lock sequence

The request-decision route begins a PostgreSQL transaction and calls `actOnJoinRequest`. The current implementation follows this sequence:

1. Read the request's rider identifier.
2. Lock that student row with `SELECT ... FOR UPDATE`.
3. Lock the parent ride row with `SELECT ... FOR NO KEY UPDATE`.

4. Lock and re-read the join-request row with `FOR NO KEY UPDATE`.
5. Re-check authorization, request state, ride state/lock time, and capacity using the current active-seat rows.
6. Apply the seat/request/event/notification changes in the same transaction.
7. Commit successful state transitions; roll back rejected transitions and exceptions.

The critical capacity check in the acceptance branch occurs after the ride lock is acquired: the application reads active seats and refuses acceptance when their count is already at `capacity_snapshot`. The successful path inserts or reactivates the rider row, changes the request to `accepted`, records its event, and performs related notifications and fare/itinerary refreshes before the route commits.

The leave branch takes the same user/ride/request locks. It marks the rider row as left, changes the request to `left`, records the event, updates notifications and refreshes the affected fare/itinerary state in the same transaction. As both operations lock the same ride row before reading and changing occupancy, competing operations on one ride are serialized. A transaction that obtains the lock later observes the committed occupancy change and revalidates against it.

3.2 Database-level guard

A normal PostgreSQL `CHECK` constraint cannot safely express an aggregate over rows in a different table. The schema therefore now also installs a trigger on activation of a `riders` row. It locks the parent ride row with `FOR NO KEY UPDATE`, counts active occupants, and rejects an insert/reactivation that would exceed capacity using SQLSTATE `23514` and constraint name `riders_ride_capacity`. A second trigger rejects a capacity reduction below the current active-occupant count.

This guard protects direct SQL writes and future write paths as well as the current API. It complements, rather than replaces, the transaction-level check. The API still performs its business validation and returns the appropriate application error; the database trigger is the last line of defence against an over-capacity write.

3.3 What the schema does and does not guarantee

The schema's `capacity_snapshot > 0` check validates that the stored capacity is positive; by itself it does not compare capacity with the number of active `riders`. The new trigger provides that cross-row enforcement. The unique constraint on `riders(ride_id,user_id)` prevents duplicate historical seat rows for the same rider and ride; the `left_at` column distinguishes active from departed occupants.

4. Race scenarios and expected outcomes

Scenario	Hazard without serialization	Expected result with current design
Two acceptances compete for the last seat	Both transactions read one free seat and both insert an active occupant.	The ride-row lock serializes the checks. One acceptance commits; the next sees a full ride and is rejected.
Acceptance races with leave	A stale occupancy read can cause an over-allocation or a lost seat update.	Both actions serialize on the ride row. The later action re-reads occupancy after the earlier commit.
Two leaves race on one request	Duplicate leave events or inconsistent request/seat state.	The request and ride locks serialize the actions; the first transition commits and the later action sees a non-accepted request.

Acceptance races with ride lock	A rider may be accepted after membership and fare shares have been frozen.	Both operations use the ride lock. Whichever commits first determines whether the acceptance is valid; after lock, normal acceptance is refused.
Direct SQL seat insert races with another insert	Application-only validation can be bypassed by another writer.	The database trigger takes the ride-row lock and checks the active count before allowing either insert.

5. Experiment methodology

5.1 Reproducible test

The integration test is `apps/web/test/integration/concurrency/last-seat.int.test.ts`. It invokes the real Next.js route handlers using the repository's integration-test request harness and a disposable local PostgreSQL test database. It does not mock the database or replace the request-decision logic.

Each measured round begins with an auto ride of capacity three: the owner and one accepted rider occupy two seats, leaving exactly one available seat. The test dispatches 50 operations together: 49 distinct riders attempt acceptance and the current accepted rider attempts to leave. The test runs 100 rounds. When the leave obtains the ride lock first, two acceptance attempts may use the two seats that become free; one of those new occupants is then removed serially to restore the one-free-seat starting condition for the next round. This serial cleanup is outside the 50 operations measured in that round.

The test uses a one-rupee fare and a `from_hub` ride so the request snapshot does not change materially as occupants leave or join. This isolates the capacity race from the separate reconfirmation policy. Each round uses 49 distinct requester identities; pending requests are reused where possible and recreated after a rider has left.

After each route operation resolves, the test reads the owner's actual application view with `getRideView(ride.id, owner.id)`. It takes the occupant count from the returned `view.occupants` array and compares the application's displayed `view.seatsLeft` against `view.capacity - view.occupants.length`. This checks the seat count exposed by the ride page rather than recomputing the seat count inside the test itself. For every read, the test also checks that the occupant count does not exceed capacity. At the end of each round, after all 50 concurrent operations and any serial cleanup have completed, it verifies that the view has two occupants and one seat left in the capacity-three ride.

The test also includes a separate direct-database-writer race: two inserts compete for the final seat and exactly one must be rejected by the database trigger.

5.2 Concurrency interpretation and limits

The 50 route-handler invocations are launched together with `Promise.allSettled`; the integration harness invokes handlers directly rather than making network requests to a running HTTP server. The application database pool is configured with `max: 10`. Consequently, this experiment creates 50 concurrent application-level operations, but at most ten connections from this pool can be active at once. It must not be described as a 50-database-session test.

The test checks committed snapshots after route calls resolve. Because the observation query uses the same ten-connection pool, it can be scheduled after additional concurrent transactions have committed; these snapshots are not a linear trace of every individual commit. The trigger provides write-time enforcement independently of this sampling. The test is a repeatable correctness stress test of the actual request handlers and database, not a latency benchmark or a proof about all possible schedules.

The repository's pull-request CI also provisions PostgreSQL 16 and runs the complete integration suite. The new test therefore participates in that database-backed CI job; its result should be recorded from the actual check run rather than inferred from the source code.

5.3 Run instructions

Run from `apps/web` after configuring the disposable local PostgreSQL database exactly as described in `apps/web/setup.md`. The integration harness deliberately refuses to reset a database unless its name ends in `_test`, it is local, and `ALLOW_TEST_DATABASE_RESET` matches that database name.

```
npm run test:integration -- test/integration/concurrency/last-seat.int.test.ts
```

The experiment prints a JSON summary prefixed by `[NFR-RD-05 concurrency results]`. Preserve that output with the test-run log when recording evidence. The direct-writer test is included in the same file.

6. Results

Execution status: passed locally on 4 November 2026. The complete PostgreSQL-backed integration suite passed: 16 test files and 112 tests, with zero failures, in 53.27 seconds. The dedicated NFR-RD-05 test also passed both of its tests in that run.

The emitted result for the full-suite run was:

```
{
  "iterations": 100,
  "concurrentOperationsPerRound": 50,
  "concurrentAcceptanceAttempts": 4900,
  "concurrentLeaveOperations": 100,
  "totalConcurrentOperations": 5000,
  "acceptedInConcurrentRounds": 170,
  "serialCleanupLeaves": 70,
  "maximumObservedOccupants": 3,
  "capacity": 3,
  "invariantViolations": 0
}
```

Thus, 5,000 mixed route-handler operations were launched in 100 rounds. The ride never exceeded its capacity of three occupants, all measured occupancy snapshots satisfied the seat/request invariants, and the round-end state was restored to two occupants with one seat available. The separate direct-database-writer test also passed: exactly one of two competing inserts for the final seat succeeded, the other was rejected with SQLSTATE `23514`, and reducing capacity below active occupancy was rejected.

A separate targeted run also passed the same two tests. Its race-dependent counters differed (166 concurrent acceptances and 66 serial cleanup leaves), as expected from lock acquisition order; the invariant results were unchanged.

7. Concurrency strategy alternatives

Strategy	Advantages	Trade-offs for this workflow
Pessimistic row locking (implemented)	Simple reasoning for a small, highly contended resource; the capacity check and state transition share one transaction; no retry protocol is required for ordinary contention.	Requests for the same ride wait behind its row lock. A long transaction reduces throughput for that ride, so transactions must remain short and must not perform slow external work while holding the lock.

Optimistic version check	Avoids holding a lock while reading; can perform well when conflicts are rare.	Requires a version column or conditional update, conflict detection, and bounded retries. A last-seat race is a high-conflict workload, so many callers may retry and still need a final atomic capacity check.
Advisory locks	Can serialize a logical ride identifier without relying on a particular row lock.	The application must define and consistently acquire the lock key on every write path. Advisory locks are easier to omit accidentally and are less visible in ordinary row-level data operations.
Serializable isolation	The database detects executions that cannot be serialized and aborts transactions that conflict.	Serialization failures require retry handling. It may abort unrelated work and adds complexity without removing the need to define transaction boundaries and handle retries correctly.
Queue serialization	A single worker or partitioned queue can process changes for one ride in order.	Adds queue infrastructure, asynchronous completion semantics, operational monitoring, and recovery/replay concerns. It is more infrastructure than this synchronous request workflow needs.

Pessimistic row locking is appropriate here because the invariant is tied to one ride, the contended operation is short, and the product needs an immediate accept/reject response. The database trigger closes the bypass around the application transaction. If contention becomes a measured throughput problem, optimistic retries or partitioned queue processing can be evaluated against workload evidence rather than adopted pre-emptively.

8. Limitations and follow-up

1. Retain the local integration-test output as execution evidence for the D3 submission.
2. Re-run the integration suite after any subsequent changes to the ride/request transaction or the capacity triggers.
3. The current pool limit is ten connections. A separate deployment/load test is required before making claims about 50 concurrent database sessions or production throughput.
4. The test does not simulate process crashes, multiple application hosts, network latency, or a production reverse proxy.
5. The report covers seat capacity for a ride. Physical vehicle double-booking is a separate invariant and is not demonstrated by this experiment.

9. Conclusion

The application serializes acceptance and leave decisions on the parent ride row and revalidates occupancy inside the transaction. The schema-level trigger extends that protection to direct database writers. The 100-round local PostgreSQL experiment and the complete integration suite passed, with zero observed capacity-invariant violations. This is correctness evidence for the implemented route-handler and database path; it is not a 50-database-session load test or a claim about production throughput.